# Manual de programador

Como primer paso esta la clase main la cual se llama proyecto2

```java
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Main.java to edit this template
 */
package proyecto2;
import java.awt.BorderLayout;
import java.awt.CardLayout;
import java.awt.Color;
import java.awt.GridLayout;
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
/**
 *
 * @author elven
 */
public class Proyecto2 extends JFrame {
    public CardLayout Interfaz = new CardLayout();
    public JPanel mainPanel = new JPanel(Interfaz);
    public JPanel pestaña1 = new JPanel(); //Menu principal
    public JPanel pestaña2 = new JPanel(new GridLayout (1,2)); //Venta de videojuegos
    public JPanel Catalogo = new JPanel(new GridLayout (0,3,10,10)); //subdivision
    public JPanel pestaña3 = new JPanel(new BorderLayout());
    public JPanel Carrito = new JPanel(new GridLayout(3,1)); //subdivision
    public JPanel Tarjetas = new JPanel(); //tarjetas del catalogo
    public JPanel Buscar = new JPanel(); //borde superior
    public JPanel JuntarF = new JPanel(new BorderLayout()); //unir botones para buscar y el scroll
    public JPanel Album = new JPanel(new GridLayout(4,6,10,10));
    public JPanel botones = new JPanel();

    public JTextField Buscar2 = new JTextField(15);

    JScrollPane Scroll1 = new JScrollPane(Catalogo);

    String[]columnaCarrito = {"Nombre","Precio","Cantidad disponible"};
    public DefaultTableModel modeloCarrito = new DefaultTableModel(columnaCarrito,0);
    public JTable TablaCarrito = new JTable(modeloCarrito);
    JScrollPane JSCarrito = new JScrollPane(TablaCarrito);

    String[]columnaHistorial = {"Codigo","Juego","Precio"};
    public DefaultTableModel modeloHistorial = new DefaultTableModel(columnaHistorial,0);
    public JTable TablaHistorial = new JTable(modeloHistorial);
    JScrollPane JSHistorial = new JScrollPane(TablaHistorial);
```

donde se generaron todos los componentes para la interfaz gráfica

```java
JButton btn1 = new JButton("Tienda de VideoJuegos");
JButton btn2 = new JButton("Regresar al menu");
JButton btn3 = new JButton("Agregar al Carrito");
JButton btn4 = new JButton("Pagar");
JButton btn5 = new JButton("Ir a album");
JButton btn6 = new JButton("Agregar Carta");
JButton btn7 = new JButton("Bucar");
JButton btn8 = new JButton("Intercambiar");

public JComboBox<String> Generos = new JComboBox<>(new String[]{"Todos","Accion","RPG","Estrategia","Deportes","Terro
public JComboBox<String> Plataforma = new JComboBox<>(new String[]{"Todas","PC","Playstation","Xbox","Nintendo Switch

public JTextField Nombres = new JTextField(15);

RepositorioCatalogoJuegos Juegos = new RepositorioCatalogoJuegos();
public listaCarrito Lista1 = new listaCarrito();

public String[][] matrizBuscar = Juegos.obtenerTodosLosJuegos();
private ModificarTienda modificador;

private AlbumMundial logica;
private int paginaVisual = 0;

public int paginaActual = 0;

Color Cpestaña1 = new Color(255,100,100);

Proyecto2(){
    pestaña1.setBackground(Color.blue);
    pestaña2.setBackground(Color.CYAN);
    Catalogo.setBackground(Color.DARK_GRAY);
    Carrito.setBackground(Color.ORANGE);
    Album.setBackground(Color.darkGray);
    Catalogo.setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));

    Scroll1.getVerticalScrollBar().setUnitIncrement(16);
    Scroll1.setVerticalScrollBarPolicy(JScrollPane.VERTICAL_SCROLLBAR_ALWAYS);
```

```java
modificador = new ModificarTienda(this);
String[][] matrizJuegos = Juegos.obtenerTodosLosJuegos();
logica = new AlbumMundial();

pestaña1.add(btn1);
pestaña1.add(btn5);
pestaña3.add(Album,BorderLayout.CENTER);
botones.add(btn6);
botones.add(btn7);
botones.add(Buscar2);
botones.add(btn8);
pestaña3.add(botones,BorderLayout.NORTH);
JuntarF.add(Buscar, BorderLayout.NORTH);
JuntarF.add(Scroll1, BorderLayout.CENTER);

Carrito.add(JSHistorial);
Carrito.add(JSCarrito);
pestaña2.add(JuntarF);
pestaña2.add(Carrito);
Carrito.add(btn4);
//Carrito.add(btn2);

mainPanel.add(pestaña1,"pestaña1");
mainPanel.add(pestaña2,"pestaña2");
mainPanel.add(pestaña3,"Album");

btn1.addActionListener((e)->{
    Interfaz.show(mainPanel,"pestaña2");
});

btn2.addActionListener((e)->{
    Interfaz.show(mainPanel,"pestaña1");
});

btn4.addActionListener((e)->{
    modificador.Pagar();
});
```

se agregan los componentes a los distintos paneles de la interfaz

```java
        btn1.addActionListener((e)->{
            Interfaz.show(mainPanel,"pestaña2");
        });

        btn2.addActionListener((e)->{
            Interfaz.show(mainPanel,"pestaña1");
        });

        btn4.addActionListener((e)->{
            modificador.Pagar();
        });

        btn5.addActionListener((e)->{
            Interfaz.show(mainPanel,"Album");
            if (paginaActual < 4) { // Suponiendo 5 páginas (0 a 4)
            paginaActual++;
            logica.mostrarPaginaEnPanel(Album, paginaActual);
    }
        });

        btn6.addActionListener((e)->{
            String codigo = JOptionPane.showInputDialog(this, "Ingrese codigo carta:");

    if (codigo != null && !codigo.trim().isEmpty()) {

        Carta nuevaCarta = logica.buscarCarta(codigo.trim().toUpperCase());

        if (nuevaCarta != null) {
            boolean exito = logica.agregarCartaEnPrimerVacio(paginaActual, nuevaCarta);

            if (exito) {
                // 4. Si se pegó, actualizamos el panel para que aparezca la imagen
                logica.mostrarPaginaEnPanel(Album, paginaActual);
                logica.guardarProgreso("Usuario1");
                JOptionPane.showMessageDialog(this, "¡Carta " + nuevaCarta.getNombre() + " pegada con éxito!");
```

se generan los eventos de los botes y los eventos para los botones del album

```java
                logica.guardarProgreso("Usuario1");
                JOptionPane.showMessageDialog(this, "¡Carta " + nuevaCarta.getNombre() + " pegada con éxito!");
            } else {
                JOptionPane.showMessageDialog(this, "La página actual está llena. Ve a otra página.", "Álbum Lleno", JOpt
            }
        } else {
            JOptionPane.showMessageDialog(this, "Código no encontrado en la tienda.", "Error", JOptionPane.ERROR_MESSAGE)
        }
    }
    });

        btn7.addActionListener((e)->{
            String criterio = Buscar2.getText();
            logica.mostrarPaginaConBusqueda(pestaña3, paginaActual, criterio);
        });

        btn8.addActionListener((e)->{
            try {
        int f1 = Integer.parseInt(JOptionPane.showInputDialog("Fila origen (0-3):"));
        int c1 = Integer.parseInt(JOptionPane.showInputDialog("Columna origen (0-5):"));
        int f2 = Integer.parseInt(JOptionPane.showInputDialog("Fila destino (0-3):"));
        int c2 = Integer.parseInt(JOptionPane.showInputDialog("Columna destino (0-5):"));

        logica.intercambiarCartas(paginaActual, f1, c1, f2, c2);
        logica.mostrarPaginaEnPanel(Album, paginaActual);
    } catch (Exception ex) {
        JOptionPane.showMessageDialog(this, "Coordenadas inválidas.");
    }
        });

        Generos.addActionListener((e)->modificador.filtrar());
        Plataforma.addActionListener((e)->modificador.filtrar());

        Nombres.getDocument().addDocumentListener(new javax.swing.event.DocumentListener(){
            public void insertUpdate(javax.swing.event.DocumentEvent e) {modificador.filtrar();}
            public void removeUpdate(javax.swing.event.DocumentEvent e) {modificador.filtrar();}
            public void changedUpdate(javax.swing.event.DocumentEvent e) {modificador.filtrar();}
        });

        modificador.filtrar();
```

```java
            int f1 = Integer.parseInt(JOptionPane.showInputDialog("Fila origen (0-3):"));
            int c1 = Integer.parseInt(JOptionPane.showInputDialog("Columna origen (0-5):"));
            int f2 = Integer.parseInt(JOptionPane.showInputDialog("Fila destino (0-3):"));
            int c2 = Integer.parseInt(JOptionPane.showInputDialog("Columna destino (0-5):"));

            logica.intercambiarCartas(paginaActual, f1, c1, f2, c2);
            logica.mostrarPaginaEnPanel(Album, paginaActual);
        } catch (Exception ex) {
            JOptionPane.showMessageDialog(this, "Coordenadas inválidas.");
        }
            });

            Generos.addActionListener((e)->modificador.filtrar());
            Plataforma.addActionListener((e)->modificador.filtrar());

            Nombres.getDocument().addDocumentListener(new javax.swing.event.DocumentListener(){
                public void insertUpdate(javax.swing.event.DocumentEvent e) {modificador.filtrar();}
                public void removeUpdate(javax.swing.event.DocumentEvent e) {modificador.filtrar();}
                public void changedUpdate(javax.swing.event.DocumentEvent e) {modificador.filtrar();}
            });

            modificador.filtrar();
            add(mainPanel);
            setSize(1050,750);
            setDefaultCloseOperation(EXIT_ON_CLOSE);
            setVisible(true);

        }
        public static void main(String[] args) {
            new Proyecto2();
        }


    }
```

el modificador es una objeto de la clase modificarTienda la cual es la encargada de modificar la interfaz de usuario en el espacio de la tienda

```java
10  import javax.swing.JDialog;
11  import javax.swing.*;
12
13  /**
14   *
15   * @author elven
16   */
17  public class ModificarTienda extends JDialog{
18      private Proyecto2 ventana;
19      RepositorioHistorial repoHistorial = new RepositorioHistorial();
20
21      public ModificarTienda(Proyecto2 ventana){
22          this.ventana = ventana;
23      }
24      public void filtrar()
25      {
26          String nombre, Genero, precio;
27          ventana.Catalogo.removeAll();
28
29          String busqeuda = ventana.Nombres.getText();
30          String genero = ventana.Generos.getSelectedItem().toString();
31          String plataforma = ventana.Plataforma.getSelectedItem().toString();
32
33          String[][]ParametrosBuscados = Busqueda.Filtros(ventana.matrizBuscar, busqeuda, genero, plataforma);
34          for(int j=0;j<ParametrosBuscados.length; j++)
35          {
36              ventana.Catalogo.add(TarjetaVisual(ParametrosBuscados[j]));
37          }
38          ventana.Catalogo.revalidate();
39          ventana.Catalogo.repaint();
40      }
41
42      public JPanel TarjetaVisual(String [] Llamar)
43      {
44          JPanel tarjeta = new JPanel();
45          JLabel Titulo = new JLabel(Llamar[1]);
46          JLabel Precio = new JLabel(Llamar[3]);
47          JLabel Genero = new JLabel (Llamar[2]);
48          JButton btn3 = new JButton("Agregar al carrito");
49
50          tarjeta.setLayout(new BoxLayout(tarjeta, BoxLayout.Y_AXIS));
```

Find:juntarF  Previous  Next  Select  2 matches

La alse modificar tienda se encarga de seleccionar los datos inscritos en las tarjetas visuales de cada carta

```java
40      }
41
42      public JPanel TarjetaVisual(String [] Llamar)
43      {
44          JPanel tarjeta = new JPanel();
45          JLabel Titulo = new JLabel(Llamar[1]);
46          JLabel Precio = new JLabel(Llamar[3]);
47          JLabel Genero = new JLabel (Llamar[2]);
48          JButton btn3 = new JButton("Agregar al carrito");
49
50          tarjeta.setLayout(new BoxLayout(tarjeta, BoxLayout.Y_AXIS));
51          tarjeta.setBackground(Color.WHITE);
52
53          tarjeta.add(Titulo);
54          tarjeta.add(Precio);
55          tarjeta.add(Genero);
56
57          btn3.addActionListener(l -> {
58          String Nombre=Llamar[1];
59          String precio=Llamar[3];
60          String Cantidad=Llamar[5];
61
62          double acumulado=0.0;
63          int Cant = 0;
64          try{
65              acumulado = Double.parseDouble(precio);
66              Cant = Integer.parseInt(Cantidad);
67          }catch(Exception e){
68              System.out.println("No se pudo ver el precio");
69          }
70          ventana.Lista1.insertar(Nombre, acumulado, Cant);
71          ventana.Lista1.RehacerTabla(ventana.modeloCarrito);
72          });
73
74          tarjeta.addMouseListener(new java.awt.event.MouseAdapter() {
75              @Override
76              public void mouseClicked(java.awt.event.MouseEvent e) {
77                  new MostrarDatos(ventana, Llamar);
78              }
79          });
80
```

Find:juntarF  Previous  Next  Select  2 matches

```java
public void Pagar()
{
    Nodo actual = ventana.Lista1.getCabeza();
    String ArticulosVacios="";
    int Cant = 0;
    double total=0.0;


    if(actual==null){
        JOptionPane.showMessageDialog(ventana, "El carrito esta vacio");
        return;
    }

    while(actual!=null)
    {
        boolean compraExistosa=false;
        for (int i = 0; i < ventana.matrizBuscar.length; i++) {
            if (ventana.matrizBuscar[i][1].equals(actual.Nombre)) {
                int stockDisponible = Integer.parseInt(ventana.matrizBuscar[i][5]);
                if (stockDisponible > 0) {
                    ventana.matrizBuscar[i][5] = String.valueOf(stockDisponible - 1);
                    total += actual.Precio;
                    Cant++;
                    compraExistosa = true;
                }
                break;
            }
        }
        if (!compraExistosa) {
            ArticulosVacios +=actual.Nombre + "\n";
        }
        actual = actual.siguiente;
    }

    if (Cant > 0) {
        DateTimeFormatter formato = DateTimeFormatter.ofPattern("dd/MM/yyyy HH:mm:ss");
        String timestamp = formato.format(LocalDateTime.now());
        String Historialtotal = String.format("%.2f", total);
        String HistorialCant = Cant + " juegos";
        repoHistorial.registrarVenta(timestamp, HistorialCant, Historialtotal);
```

ntarF  Previous  Next  Select  2 matches

26:33  INS  Windows (CR

...M]  Proyecto2.java [-/M]  Busqueda.java  ModificarTienda.java  MostrarDatos.java  Nodo.java  RepositorioHistorial.java  list...

Source  History

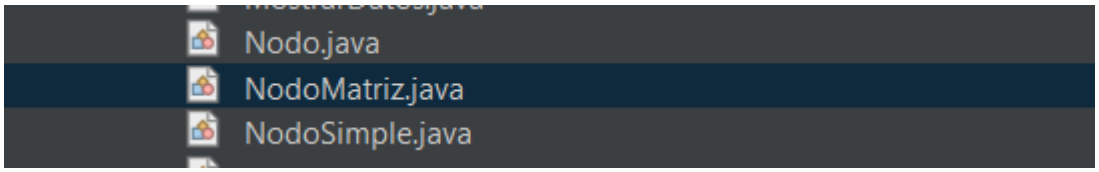
```java
                    }
                    break;
                }
            }
            if (!compraExistosa) {
                ArticulosVacios +=actual.Nombre + "\n";
            }
            actual = actual.siguiente;
        }

        if (Cant > 0) {
            DateTimeFormatter formato = DateTimeFormatter.ofPattern("dd/MM/yyyy HH:mm:ss");
            String timestamp = formato.format(LocalDateTime.now());
            String Historialtotal = String.format("%.2f", total);
            String HistorialCant = Cant + " juegos";
            repoHistorial.registrarVenta(timestamp, HistorialCant, Historialtotal);
            ventana.modeloHistorial.addRow(new Object[]{timestamp,"cantidad Juegos: "+ Cant, total});

        }

        if (!ArticulosVacios.isEmpty()) {
            JOptionPane.showMessageDialog(ventana, "Ya no hay unidade");
        } else {
            JOptionPane.showMessageDialog(ventana, "Se ha realizado su compra");
        }
        ventana.Lista1.vaciar();
        ventana.Lista1.RehacerTabla(ventana.modeloCarrito);
        filtrar();
        ventana.Catalogo.revalidate();
        ventana.Catalogo.repaint();
    }
}
```

generar las distintas cartas y realizar todo la creación del pagar asignar espacios de las cartas y el llamar al a clase mostrardatos para que esta en el momento en el que se haga click se muestren los datos completos de la tarjeta

```java
package proyecto2;

import javax.swing.*;
import java.awt.*;

public class MostrarDatos extends JDialog{
    public MostrarDatos(Proyecto2 ventana, String[] datos)
    {
        super(ventana, "Información del juego",true);
        setLayout(new GridLayout(8,1,10,0));
        JButton btn1 = new JButton("Regresar");
        JLabel Titulo = new JLabel(datos[1]);
        JLabel Precio = new JLabel(datos[3]);
        JLabel Genero = new JLabel (datos[2]);
        JLabel Plataforma = new JLabel(datos[4]);
        JLabel Cantidad= new JLabel(datos[5]);
        JLabel Descripcion = new JLabel (datos[6]);

        btn1.addActionListener(e->{
            dispose();
        });
        add(btn1);
        add(Titulo);
        add(Precio);
        add(Genero);
        add(Plataforma);
        add(Cantidad);
        add(Descripcion);

        this.setSize(350,350);
        this.setVisible(true);
    }
}
```

se le envia un string de la fila a la cual esta asignada los atributos de la tarjeta y genera un panel en el cual se puede apreciar los datos completos de la carta

Las clases nodo, nodo matriz y nodo simple son mismas clases que permiten la creación de una lista simple por medio de nodos



```java
package proyecto2;

/**
 *
 * @author elven
 */
public class Nodo {
    public String Nombre;
    public double Precio;
    public int cantidad;
    public Nodo siguiente;


    public Nodo(String Nombre, double Precio, int cantidad) {
        this.Nombre = Nombre;
        this.Precio = Precio;
        this.cantidad = cantidad;
        this.siguiente = null;
    }
}
```

Source History

```java
 * @author elven
 */
public class NodoMatriz {

    Carta dato;
    NodoMatriz arriba;
    NodoMatriz abajo;
    NodoMatriz izquierda;
    NodoMatriz derecha;

    int fila;
    int columna;

    public NodoMatriz(int fila, int columna) {
        this.fila = fila;
        this.columna = columna;
        this.dato = null;
        this.arriba = null;
        this.abajo = null;
        this.izquierda = null;
        this.derecha = null;
    }

    public boolean estaVacia() {
        return dato == null;
    }

    public Carta getDato() {
        return dato;
    }

    public NodoMatriz getArriba() {
        return arriba;
    }

    public NodoMatriz getAbajo() {
        return abajo;
    }

    public NodoMatriz getIzquierda() {
```

Find:juntarF Previous Next Select 2 matches

```java
*/
public class NodoSimple<T> {
    T dato;
    NodoSimple<T> siguiente;

    public NodoSimple(T dato) {
        this.dato = dato;
        this.siguiente = null;
    }

    public T getDato() {
        return dato;
    }

    public NodoSimple<T> getSiguiente() {
        return siguiente;
    }

}
```

Las clases repositorio son las encargadas de guardar y leer la información de un archivo txt

```java
public class RepositorioCatalogoJuegos {

    private String nombreArchivo = "data/catalogo.txt";

    public RepositorioCatalogoJuegos() {
        asegurarArchivoExiste();
    }

    public void asegurarArchivoExiste() {
        File archivo = new File(nombreArchivo);
        System.out.println(archivo.getAbsolutePath());
        if (!archivo.exists()) {
            try {
                archivo.createNewFile();
            } catch (IOException e) {
                System.out.println("Error al crear el archivo: " + e.getMessage());
            }
        }
    }

    // Retorna una matriz [n][2] donde [i][0] es usuario y [i][1] es codigo del juego
    public String[][] obtenerTodosLosJuegos() {
        int cantidadLineas = contarLineas();
        String[][] matrizUsuarios = new String[cantidadLineas][7];

        try {
            BufferedReader lector = new BufferedReader(new FileReader(nombreArchivo));
            String linea;
            int indice = 0;
            while ((linea = lector.readLine()) != null) {
                if (linea.contains(",")) {
                    String[] partes = linea.split(",");
                    matrizUsuarios[indice][0] = partes[0]; //codigo
                    matrizUsuarios[indice][1] = partes[1]; //nombre
                    matrizUsuarios[indice][2] = partes[2]; //Genero
                    matrizUsuarios[indice][3] = partes[3]; //Precio
                    matrizUsuarios[indice][4] = partes[4]; //Plataforma
                    matrizUsuarios[indice][5] = partes[5]; //Stock
```

```java
import javax.swing.table.DefaultTableModel;
/**
 *
 * @author elven
 */
public class RepositorioHistorial {
    private String nombreArchivo = "historial.txt";

    public RepositorioHistorial(){
        asegurarArchivoExiste();
    }

    public void asegurarArchivoExiste() {
        File archivo = new File(nombreArchivo);
        System.out.println(archivo.getAbsolutePath());
        if (!archivo.exists()) {
            try {
                archivo.createNewFile();
            } catch (IOException e) {
                System.out.println("Error al crear el archivo: " + e.getMessage());
            }
        }
    }

    public String[][] obtenerHistorial() {
        int cantidadLineas = contarLineas();
        String[][] matrizHistorial = new String[cantidadLineas][3];

        try (BufferedReader lector = new BufferedReader(new FileReader(nombreArchivo))) {
            String linea;
            int indice = 0;
            while ((linea = lector.readLine()) != null) {
                if (linea.contains(",")) {
                    String[] partes = linea.split(",");
                    if (partes.length >= 3) {
                        matrizHistorial[indice][0] = partes[0];
                        matrizHistorial[indice][1] = partes[1];
                        matrizHistorial[indice][2] = partes[2];
                        indice++;
                    }
```

```java
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
        return matrizHistorial;
    }

    private int contarLineas() {
        int lineas = 0;
        try {
            BufferedReader lector = new BufferedReader(new FileReader(nombreArchivo));
            while (lector.readLine() != null) {
                lineas++;
            }
            lector.close();
        } catch (IOException e) {
            return 0;
        }
        return lineas;
    }

    public void registrarVenta(String fecha, String cantidad, String total) {
        try (PrintWriter escritor = new PrintWriter(new FileWriter(nombreArchivo, true))) {
            escritor.println(fecha + "," + cantidad + "," + total);
        } catch (IOException e) {
            System.out.println("Error al escribir historial: " + e.getMessage());
        }
    }
    public void cargarEnTabla(DefaultTableModel modelo) {
        modelo.setRowCount(0); // Limpiamos la tabla visual
        String[][] datos = obtenerHistorial();
        for (String[] fila : datos) {
            if (fila[0] != null) {
                modelo.addRow(fila);
            }
        }
    }
}
```

al igual que la clase gestor archivos

Source History

```java
public class GestorArchivos {

    private static final String RUTA_ALBUM   = "data/album.txt";
    private static final String RUTA_EQUIPO  = "data/album_equipo.txt";
    private static final String RUTA_USUARIO = "data/album_usuario.txt";
    private static final String SEP          = "\\|";

    private int contarLineas(String ruta) {
        int n = 0;
        try (BufferedReader br = new BufferedReader(new FileReader(ruta))) {
            String l;
            while ((l = br.readLine()) != null) {
                l = l.trim();
                if (!l.isEmpty() && !l.startsWith("#") && !l.startsWith("usuario|")) n++;
            }
        } catch (IOException e) {
            System.out.println("No se pudo leer: " + ruta);
        }
        return n;
    }

    public CartaUsuario[] cargarCartasUsuario() {
        verificarYCrearArchivo(RUTA_USUARIO);
    int total = contarLineas(RUTA_USUARIO);

    // Usamos un arreglo temporal del tamaño de las líneas
    CartaUsuario[] reg = new CartaUsuario[total];
    int idx = 0;

    try (BufferedReader br = new BufferedReader(new FileReader(RUTA_USUARIO))) {
        String l;
        while ((l = br.readLine()) != null) {
            l = l.trim();
            if (l.isEmpty() || l.startsWith("#")) continue;

            String[] p = l.split("\\|");

            if (p.length >= 5) {
                reg[idx++] = new CartaUsuario(
                    p[0].trim(), // Nombre de usuario
```

```java
} catch (IOException | NumberFormatException e) {
    System.err.println("Error al procesar album_usuario.txt: " + e.getMessage());
}
    CartaUsuario[] resultado = new CartaUsuario[idx];
    System.arraycopy(reg, 0, resultado, 0, idx);
return resultado;
}


public void guardarCartasUsuario(CartaUsuario[] registros, int usados) {
    try (BufferedWriter bw = new BufferedWriter(new FileWriter(RUTA_USUARIO))) {
    // Guardamos solo los datos reales, sin encabezados confusos
    for (int i = 0; i < usados; i++) {
        if (registros[i] != null) {
            bw.write(registros[i].toLinea());
            bw.newLine();
        }
    }
    bw.flush();
} catch (IOException e) {
    System.out.println("Error al guardar: " + e.getMessage());
}
}


public Carta[] cargarCartas() {
verificarYCrearArchivo(RUTA_ALBUM);

int total = contarLineas(RUTA_ALBUM);
Carta[] catalogo = new Carta[total];
int idx = 0;

try (BufferedReader br = new BufferedReader(new FileReader(RUTA_ALBUM))) {
    String linea;
    while ((linea = br.readLine()) != null) {
        linea = linea.trim();

        if (linea.isEmpty() || linea.startsWith("#")) continue;
```

RepositorioHistorial.java × listaCarrito.java × albumMalla.java × AlbumMundial.java × Carta.java × GestorArchivos.java × Generad...

History

```java
                    Integer.parseInt(datos[6].trim()), // puntosSalud
                    datos[7].trim()  // rutaImagen
                );

            } catch (NumberFormatException e) {
                System.err.println("Error en formato numérico en línea: " + linea);
            }
        }
    }
} catch (IOException e) {
    System.err.println("Error al leer el catálogo de cartas: " + e.getMessage());
}

// Ajustamos el arreglo por si hubo líneas vacías o errores
if (idx < total) {
    Carta[] resultado = new Carta[idx];
    System.arraycopy(catalogo, 0, resultado, 0, idx);
    return resultado;
}

return catalogo;
}

private void verificarYCrearArchivo(String ruta) {
try {
    File archivo = new File(ruta);
    File carpeta = archivo.getParentFile();

    if (carpeta != null && !carpeta.exists()) {
        carpeta.mkdirs();
    }

    if (!archivo.exists()) {
        archivo.createNewFile();
        System.out.println("Archivo creado automáticamente: " + ruta);
    }
} catch (IOException e) {
    System.err.println("Error crítico al crear archivo: " + e.getMessage());
}
```

y el resto de clases son para la malla ortogonal